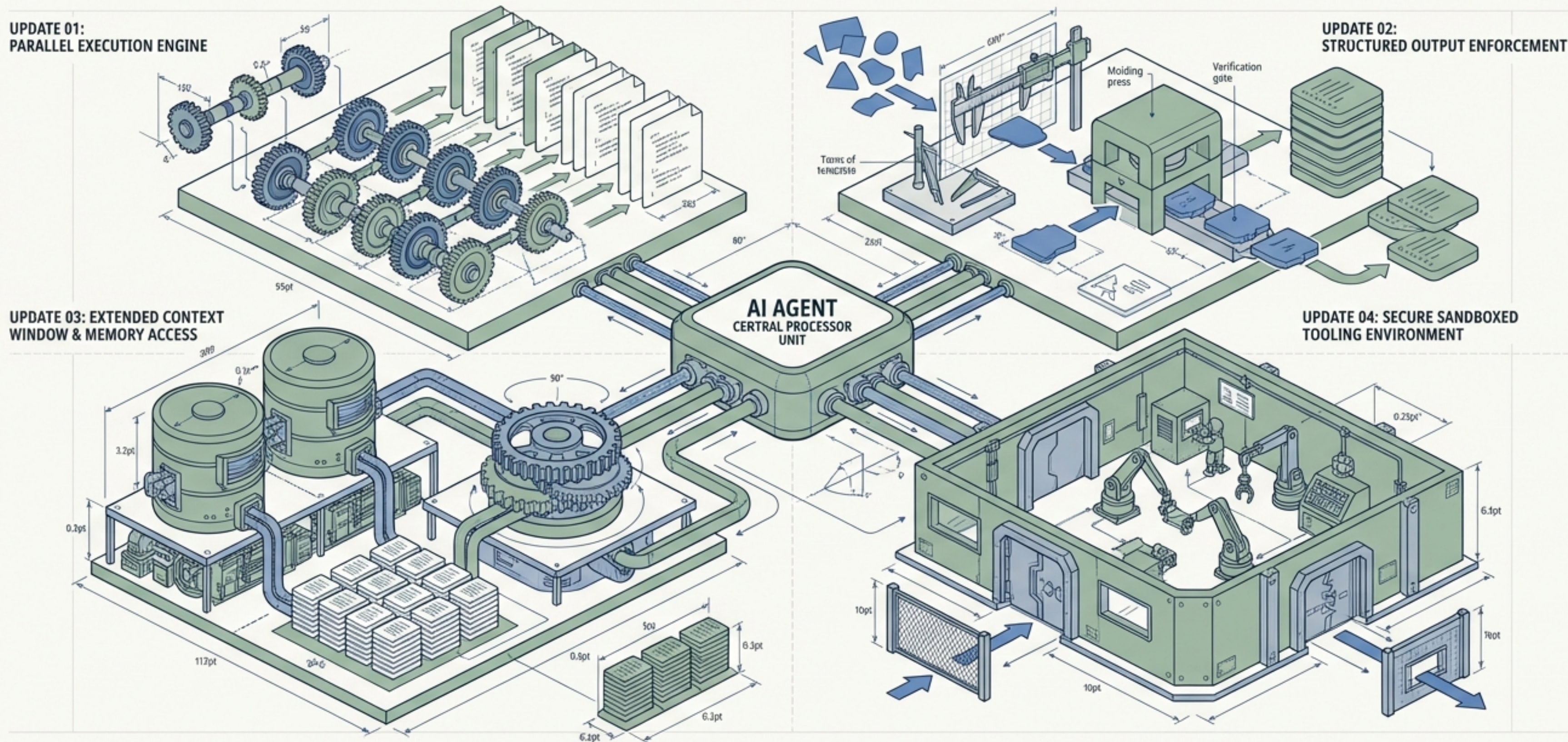


Tool Calling 2.0 来了

构建高级 AI Agent 的范式转移：解析 Anthropic 工具调用的四大核心升级



工具调用 1.0 时代的阿喀琉斯之踵

过去两年的标准范式：完全依赖 LLM 作为每一次离散步骤的决策粘合剂。



规模化困境：当任务变得复杂时发生的三大灾难

Token 浪费

网页抓取等工具返回大量无用 HTML 标签，白白消耗 Context 空间。

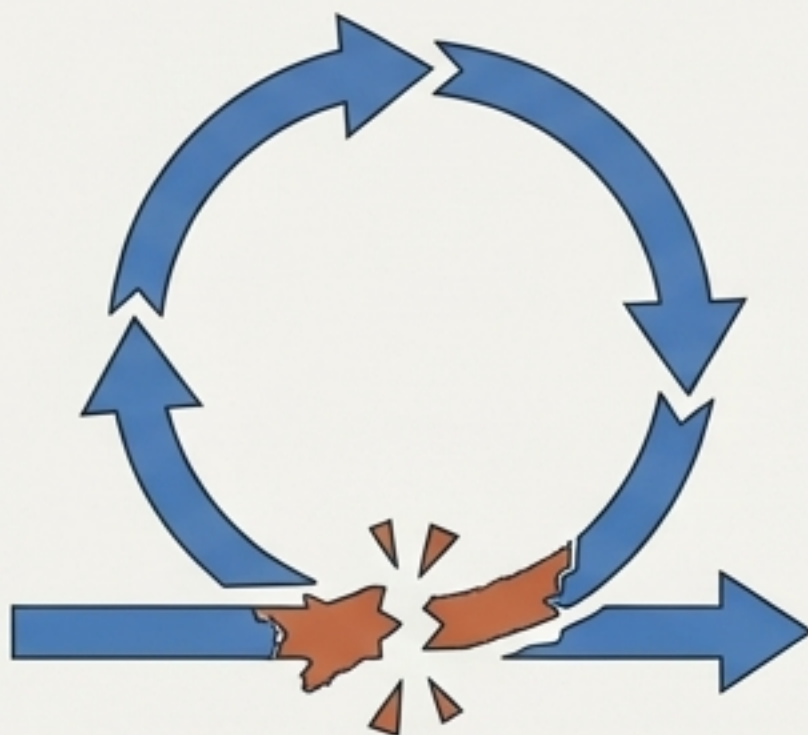
```
<div> class="code">cupFrreDetail"></div>
<div>,<span class="enent">,</div>
<span class="origest"><ss
<span>,</span><style="av
<span class="amimwky(&lt;u>
<div></div>,<span taktu
<span class="tokeuhatker
<span class="eocsuhatrier
<<div><span class="
<span>,<span class="
<span class="eocsuhatrier
<<div>,<span><
<span class="
<span class="
<div>,<span><
<span class="
<span class="
<div>,<span>
<span class="
<span class="
<div>,<span>
<span class="
</div>
```



有效数据

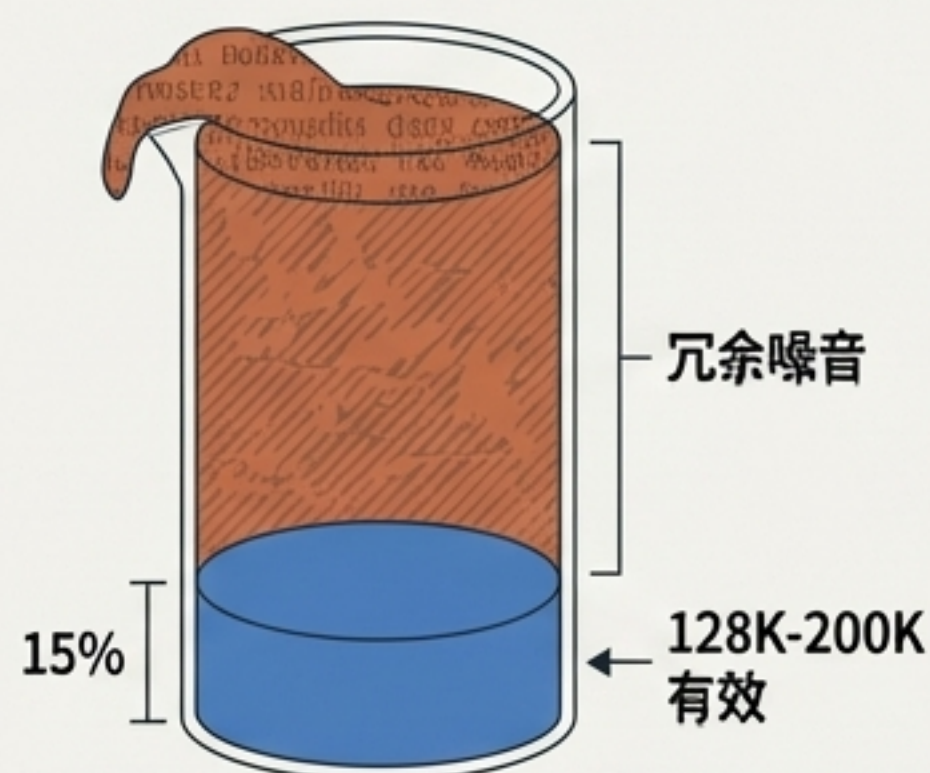
逻辑断层

依赖大模型每次重新生成 JSON 来执行 For 循环（如遍历多封邮件），极易产生幻觉和不确定性。



上下文污染

即拥有 1M 上下文，实际有效窗口仅 128K - 200K。中间步骤冗余数据导致模型遗忘关键指令。



破局与重构：欢迎来到 Tool Calling 2.0 时代

这不是简单的 API 升级，而是彻底重构工具调用的底层逻辑——从「输出数据」向「输出可执行逻辑」进化。

1. 程序化工具调用



2. 动态网页过滤



3. 工具搜索



4. 工具使用示例



核心进化：从「输出 JSON」到「输出可执行代码」

Opus 4.5 在此类任务中的 Context 占用显著缩小，成功率大幅提升。

旧范式：输出多重离散 JSON

```
{  
  "tool_choice": "fetch_data",  
  "params": {"query": "user_list"}  
}  
  
{  
  "tool_choice": "filter_active",  
  "params": {"threshold": 0.8}  
}  
  
{  
  "tool_choice": "send_email",  
  "params": {"recipient": "all"}  
}
```

LLM 作为粘合剂，暴露所有中间状态，导致严重冗余。

新范式：单次沙盒代码执行

Sandbox Environment

```
for user in data.users:  
    if user.active and user.score > 0.8:  
        send_email(user.email, content)
```

大模型直接在沙盒内编写代码，自动流转数据，仅返回最终结果。

极简接入：如何在现有 Agent 中激活程序化调用？

侵入性极小的改动，无需重构整个智能体编排。

```
# Agent configuration example
from agent_sdk import Agent, ToolRegistry

# ... existing tool definitions ...

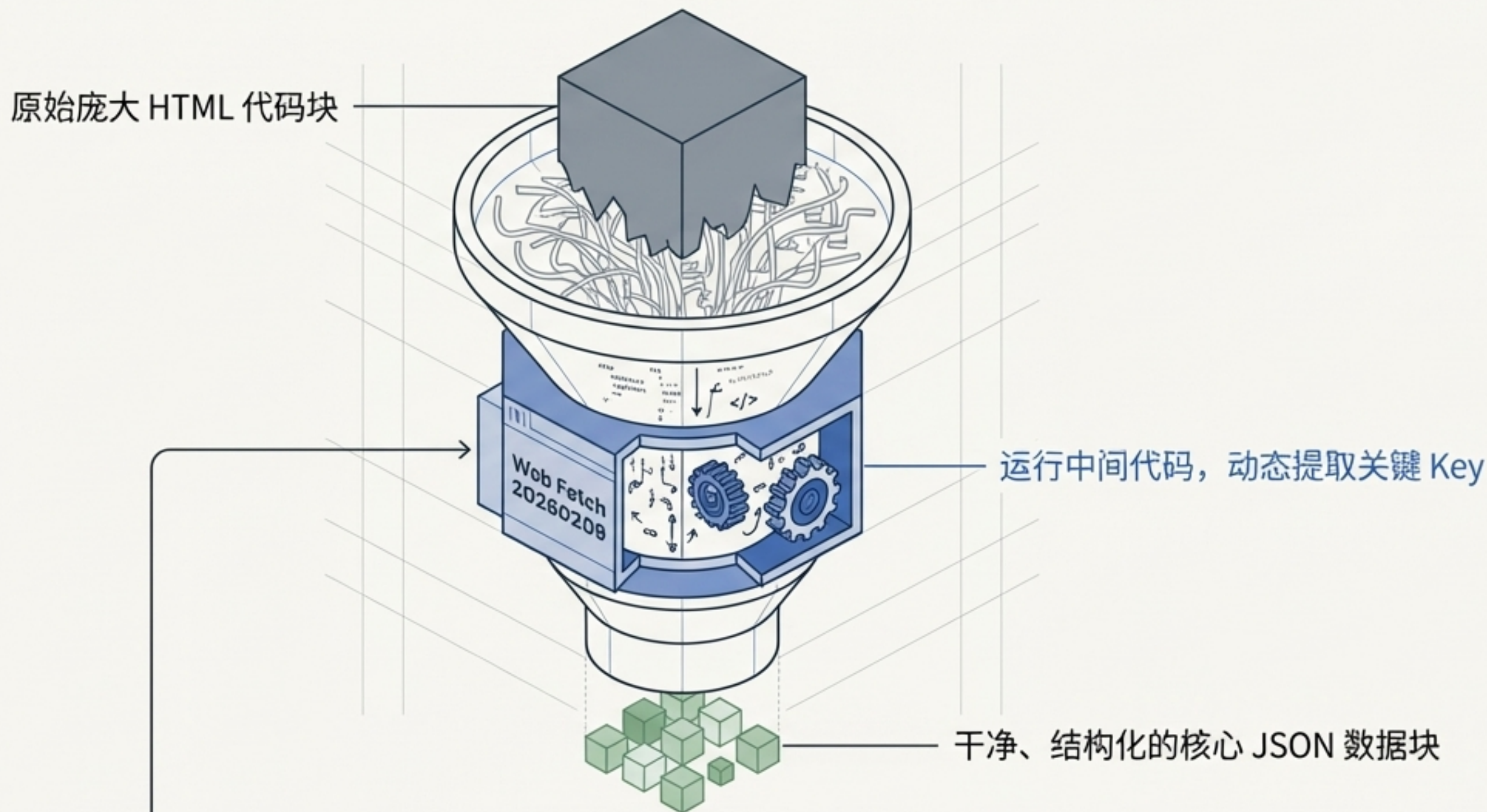
def configure_agent():
    tools = ToolRegistry()
    # ... existing tool registration ...

    # 激活程序化调用
    agent = Agent(
        name="AnalystBot",
        model="opus-4.5",
        tools=tools,
        execution_config={
            "enable_code_execution": True,
            # 核心工具注入与授权
            "tools": [
                "code_execution_20260120",
                "allowed_caller"
            ],
            "max_steps": 10
        }
    )
    return agent
```

注入核心工具：自动激活沙盒内部代码执行流。

目标工具参数授权：允许模型在生成的代码中直接调用此特定工具。

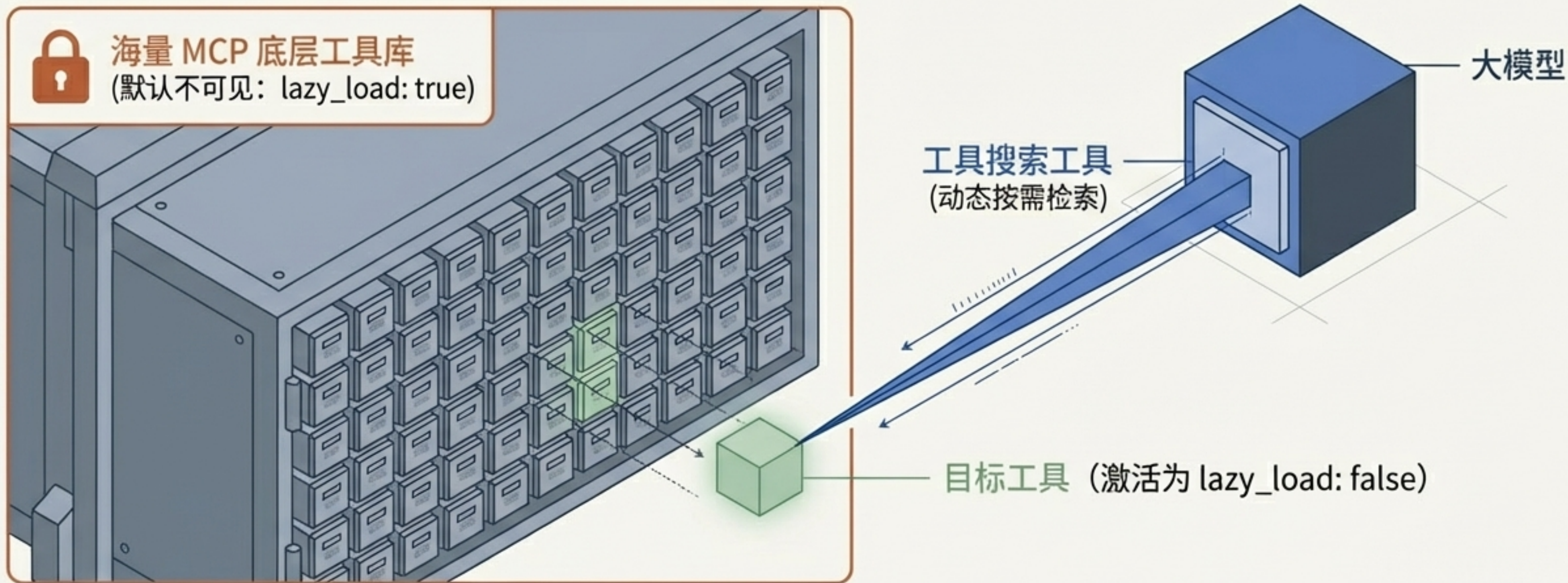
告别冗余：Web Fetch 的动态过滤 (Dynamic Filtering)



平均减少 24% Token 消耗

工具搜索：打破数百个工具的上下文诅咒

侵入性极小的改动，无需重构整个智能体编排。



成本极低：仅消耗约 **500 Token**，实现高达 **80%** 的 Context 上下文空间优化。

工具使用示例：解决复杂参数的填空难题

仅提供基础 Schema 定义

SLA 期限	
工单级别	



引入 input_examples 真实示例数组

SLA 期限	2 小时 SLA
工单级别	级别 A

```
input_examples: [  
  {  
    "SLA 期限": "2 小时 SLA",  
    "工单级别": "级别 A"  
  }  
]
```



核心成效：通过引入真实业务关联示例，复杂嵌套参数的填写准确率从 72% 跃升至 90%。

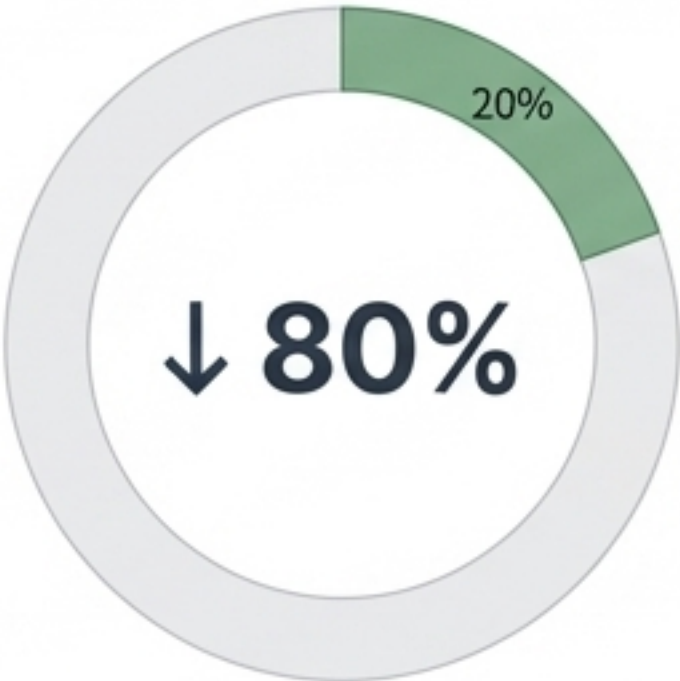
降本增效：Tool Calling 2.0 性能收益盘点

Token 消耗降低



得益于程序化调用与循环批处理

Context 空间释放



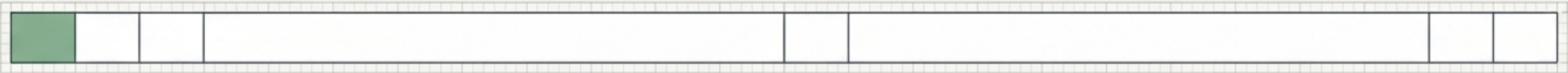
得益于工具搜索与 MCP 延迟加载

执行精准度飞跃



得益于 input_examples 机制

Context Window allocation bar



0.5pt

0.5pt

重塑智能体的构建方式

**让大模型做它最擅长的事（编写逻辑代码），
把枯燥的数据流转留在沙盒内。**

极大释放 Token 空间，赋予智能体处理长期、复杂任务的真正能力。

持续站在 AI 架构的最前沿

- ✓ 分步配置教程
- ✓ 程序化调用实战代码示例
- ✓ 每周专家技术直播
- ✓ 顶级 AI 构建者架构心得



加入 AI Builder Club

想要获取完整教程与最佳生产实践？点击视频下方描述栏链接加入。